

TECHNICAL WHITEPAPER · OCTOBER 2026

Eva

A Local-First Cognitive Workbench

Architecture, trust boundaries, and direction

A technical examination of an integrated assistant architecture: durable context, provider-neutral orchestration, evidence-bearing actions, and a user-owned control plane.

For technical leaders and sophisticated adopters
9 October 2026 · Development version 5.6.12
Source evidence: [appataalks/eva-agent · 8ef3559](#)

READER'S GUIDE

Contents

Executive summary

1. The problem and the design thesis

2. Inspiration and product identity

3. System architecture

4. Memory: ordinary, protected, and contextual

5. Native actionability and receipts

6. Workspaces, terminals, Skills, and media

7. Deployment and operational model

8. Technical analysis

9. Differentiation without hype

10. Goals and staged direction

11. Proposed evaluation program

12. Material limitations and risk register

13. Evidence appendix

14. Implementation status at the evidence snapshot

15. Governance and adoption guidance

This whitepaper distinguishes current implementation from experimental integrations and future direction. Source references point to the evidence snapshot, not to live personal runtime data.

Whitepaper date: 2026-10-09

Evidence snapshot: repository commit 8ef3559 (feat: Super Intelligence harness and contextual neural memory)

Evidence status: default-branch source snapshot, aligned with the repository's stated unreleased development version 5.6.12; not the latest published standalone release

This paper describes the product that can be substantiated from the source tree and its technical documentation. It distinguishes shipped behavior from experimental integrations and future direction. “Super Intelligence” (SI) is Eva's product name for its orchestration harness; it is not a claim of artificial general intelligence, sentience, or a scientific breakthrough.

Executive summary

Eva is a framework-free browser and Electron application for conversation, durable memory, voice, live information, automation, and software work. Its central design decision is to separate a user-facing control plane from execution systems. The browser UI owns interaction and presentation; a local Python bridge owns request normalization, memory access, policy, retrieval, provider adaptation, and receipts; Electron's main process owns privileged desktop capabilities such as terminal sessions and workspace IPC. External model and agent runtimes remain replaceable adapters rather than becoming Eva's identity.

The result is neither a generic chat shell nor an autonomous operator with unlimited reach. Eva is best understood as a consent-oriented local workbench: it gives a person one conversation and one operational surface across multiple model routes, while making context, execution, and uncertainty visible enough to review. The local-first posture is mode-dependent: SQLite, LM Studio, local speech, and loopback services can provide a no-cloud path, while OpenAI, Copilot ACP, Gemini, GitHub MCP, Kusto, web retrieval, Signal, and other integrations are optional configured dependencies.

The architecture has four important consequences.

1. **Routing is a policy problem, not a dropdown problem.** The AIG gateway can use an automatic policy or a pinned backend, with direct OpenAI, Copilot through ACP, and OpenAI-compatible local/SI Harness routes. Tool-required work fails closed when no tool-capable route is available.
2. **Memory is evidence-bearing context, not authority.** Ordinary memory is inspectable SQLite by default, with optional Kusto support and semantic recall where configured. Records have provenance, lifecycle and correction paths. Protected memory is deliberately separate, encrypted/locked, and never silently injected merely because it exists.
3. **Actionability is bounded by policy and receipts.** Browser, desktop, terminal, workspace, messaging, file, and document operations have distinct capability and consent boundaries. Eva should report a verified result, a partial result, or no verified receipt, rather than infer success from a model's prose.
4. **The product is useful today but incomplete by design.** Coding workspaces, terminals, Skills, voice, camera hooks, document abilities, bounded research, and Agent Operations are meaningful foundations. Durable research jobs, provider-neutral multi-agent adapters, durable approval/audit UI, diff review, explicit merge workflows, broader deployment portability, and several integrations remain roadmap work.

For technical leaders, Eva's differentiation is therefore architectural and operational: local ownership of state, explicit execution brokerage, source receipts, provider interchangeability, and a UI that treats memory and running work as first-class objects. It does not rest on unsupported benchmark claims, a claim of a true neural brain, or the promise that every feature works offline or on every host.

1. The problem and the design thesis

Modern assistants split a work session across incompatible surfaces. A chat product remembers some conversation but cannot safely operate a repository. A coding agent can modify files but often exposes only terminal scrollback. A local model protects data but may lack current information or a reliable tool loop. A cloud agent can search or act, but the user may not know which model saw which context, what actually happened, or whether a final sentence is evidence or speculation.

Eva addresses this fragmentation with a compact thesis:

Put the user's conversation, durable context, operational state, and consent decisions in one local control plane; treat models and execution runtimes as constrained, observable capabilities behind it.

This thesis has several practical implications.

1.1 Context should be reviewable

A memory entry is not simply a hidden prompt fragment. Eva's documented memory model exposes provenance, associations, history, corrections, and lifecycle controls. Explicit facts are committed before Eva acknowledges saving them; ordinary questions do not become facts. Recall is a request-scoped view and does not rewrite browser history. These details are product behavior, not a claim that extracted context is true: a user can state something incorrectly, an extractor can miss it, and a model can misinterpret it. The UI and data model must preserve that distinction.

1.2 Execution should be narrower than conversation

Text generation is comparatively easy to retry; a sent message, edited file, deleted record, or remote mutation is not. Eva consequently separates read/search/fetch/think consent from execute/edit/move/delete and unknown operations. Approval behavior depends on the active permission policy: interactive paths request a decision, while user-selected workspace autonomy can authorize permitted operations without a new prompt. Hard-blocked operations remain prohibited. ACP terminal requests are disabled because the protocol does not reliably correlate a terminal request with a permission decision; authorized terminal work uses Eva's own browser/desktop paths and Electron broker.

1.3 A model is a component, not the product boundary

The same Eva interaction can be routed to a direct OpenAI endpoint, Copilot CLI through Agent Client Protocol (ACP), LM Studio, or an OpenAI-compatible SI Harness endpoint. This is valuable for availability, privacy, cost, and task fit, but it also means the bridge must normalize requests, preserve response contracts, and disclose limitations. Eva's identity is the policy, memory, execution, and presentation layer—not a permanent dependency on one model vendor.

1.4 Local-first is a control posture, not a universal deployment promise

The repository supports local SQLite, local model inference, local speech dependencies, and loopback bridge endpoints. It also supports cloud AI and configured remote services. “Local first” means the local route is a legitimate default and the user controls enablement; it does not mean all features are offline, that every provider can run on an old Linux host, or that an Electron AppImage contains every external runtime.

2. Inspiration and product identity

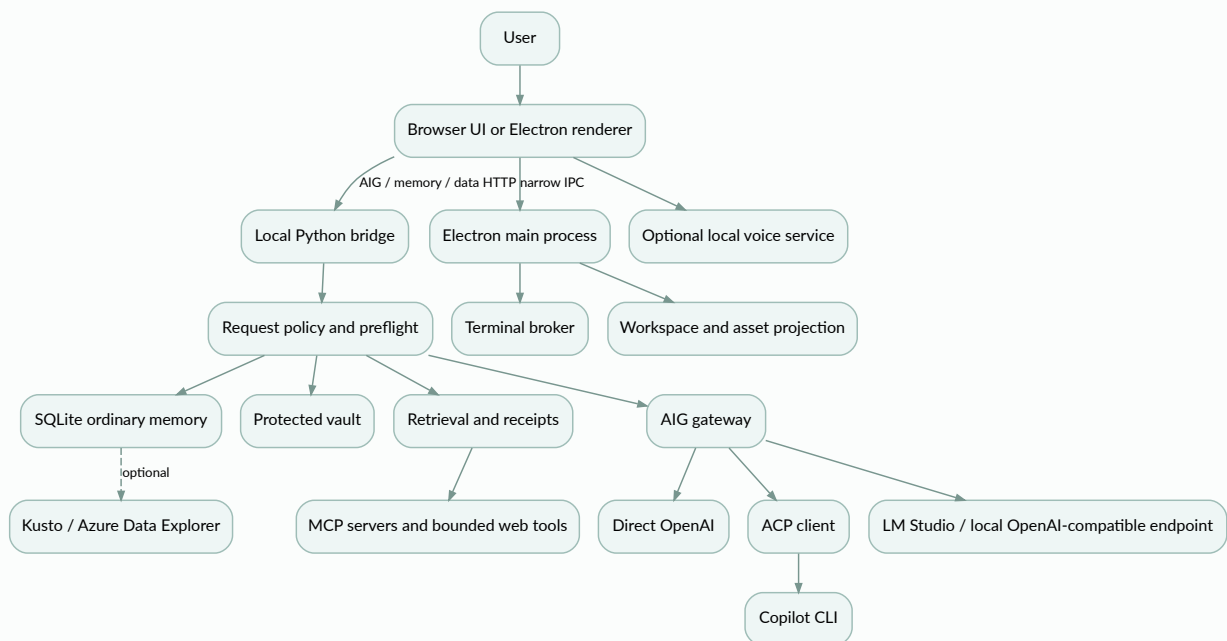
Eva's documented identity combines three kinds of inspiration without claiming ownership of their source systems.

- **A fictional ethical ideal.** The repository names Lieutenant Commander Data from *Star Trek: The Next Generation* as inspiration for curiosity, precision, ethical judgment, candid uncertainty, and care in understanding people. This is aspirational identity material, not a claim that Eva possesses fictional abilities or consciousness.
- **Evidence-centered memory research.** TencentDB Agent Memory informs concepts such as evidence, atoms, scenarios, traits, and progressive disclosure. Eva retains its own local SQLite/Kusto implementation and does not adopt TencentDB's deployment stack.
- **Coding-agent and protocol practice.** Traycer, OpenCode, Cline, Gemini CLI, Goose, Codex CLI, OpenHands, Aider, SWE-agent, MCP, ACP, Electron, xterm.js, node-pty, Git, and related systems are listed in the technical documentation as inspirations, references, protocols, or integrations with different statuses. Their mention is not an affiliation, embedded code claim, or competitive performance claim.

The interface expresses the identity through a focused chat surface, optional LCARS styling, Agent Operations, Memory Topology, Skills, Assets, Workspaces, terminals, voice views, and source-aware receipts. A lightweight confidence/connectivity visualization can help a person inspect relationships. It should be described as a pseudo-3D visualization of memory connectivity and confidence—not as a true four-dimensional neural graph or a biological brain simulation.

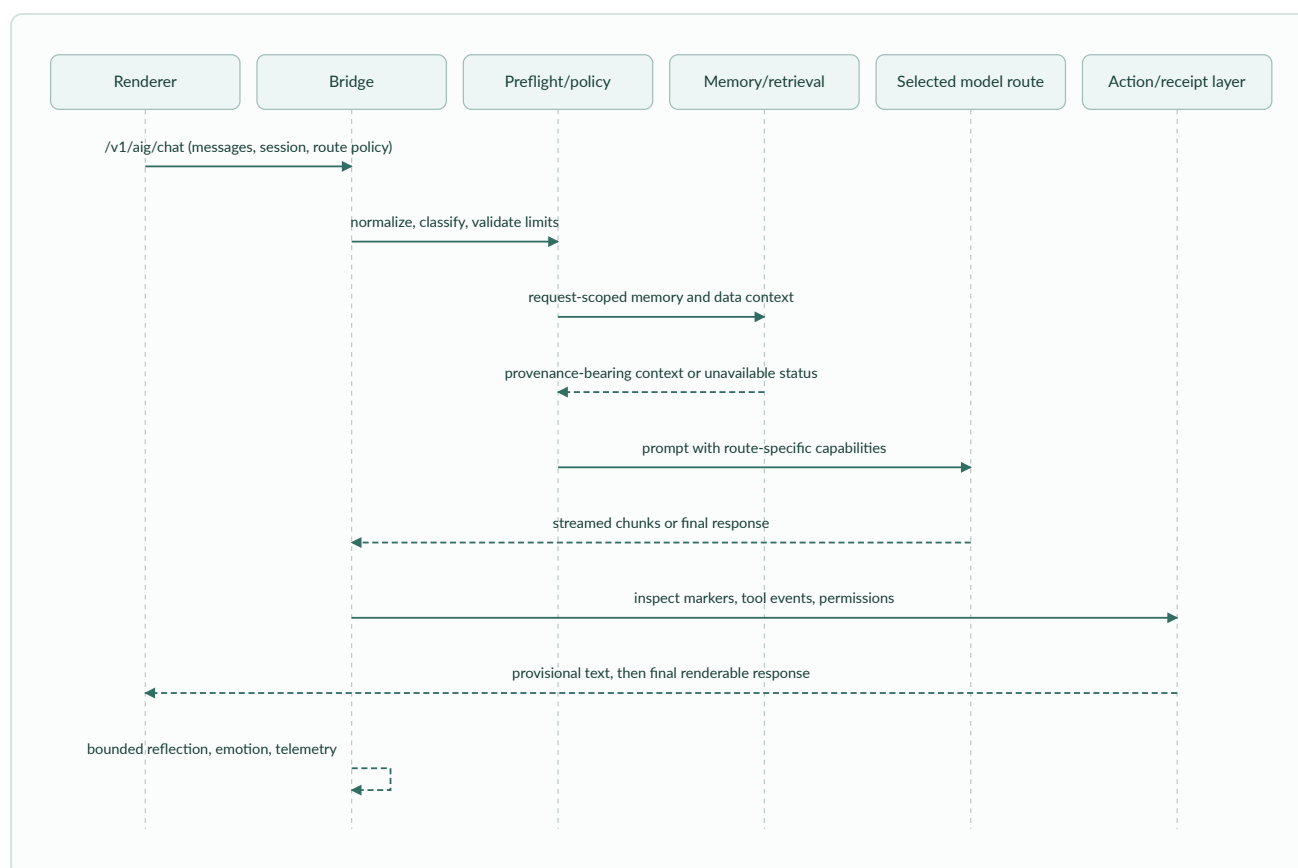
3. System architecture

3.1 Logical topology



The bridge is normally bound to loopback. It exposes OpenAI-compatible chat contracts for provider and client interoperability, while Eva-specific endpoints handle memory, mode, workspaces, telemetry, health, and capability checks. Electron starts the bridge on a free local port, supplies private capability tokens, waits for health, and quits with an error dialog if the bridge cannot start. A browser-only deployment can use a separately running bridge, but the split-deployment assumptions are not silently removed: the ACP roadmap explicitly tracks a future single-host milestone.

3.2 Request lifecycle



For streaming, the bridge emits newline-delimited JSON chunk and done events. The browser keeps provisional text-only content so action, Signal, file, and browser markers cannot execute early; it removes that bubble and renders the final response once. Bridge telemetry records timing, route, model labels, and chunk counts without prompt or response text.

3.3 AIG routing modes

The UI's top-level chat selector remains Eva/AIG. AIG's backend selector can express policy rather than forcing the user to change the conversation surface.

- **Automatic policy (auto-balanced or auto-fast).** The bridge evaluates availability, task requirements, and route constraints. A tool-required request must use ACP or local MCP-capable execution and fails closed if neither is available.
- **Pinned ACP/Copilot.** Copilot CLI is authenticated locally and connected through ACP. ACP streams session updates and may request MCP tools. Eva presents an in-chat Allow once or Reject decision, with a narrower standing consent for routine reads/searches.
- **Pinned direct OpenAI.** A configured OpenAI key is required. Eva retains its persona, memory lifecycle, response rendering, and action markers, but direct provider tool behavior is not equivalent to ACP.
- **SI Harness/local route.** For normal AIG sends, the browser submits the request to the bridge, which assembles context and calls the configured OpenAI-compatible endpoint. A legacy direct-send provider

also exists, but it is not the normal AIG routing path. No cloud model credential is required for the local-only path, but local tools can still access the network when explicitly configured.

- **OpenAI-compatible SI Harness endpoint.** SI Harness names the product's compatible orchestration/endpoint surface. Compatibility provides a request contract, not identical model quality, tool semantics, context windows, latency, or safety behavior. Eva must retain route labels and capability checks rather than treating every compatible endpoint as interchangeable.

The bridge validates model names, completion-token limits, reasoning-effort values, stream flags, and policy mode. It restricts a configurable OpenAI-compatible URL to loopback HTTP in the inspected implementation; this is a deliberate endpoint/network boundary, not a “host anywhere” guarantee. An organization that wants a remote service needs an explicit, reviewed deployment change rather than an undocumented environment-variable escape hatch.

3.4 Memory and cognition pipeline

An AIG request can pass through memory context, data retrieval, persona, and the final model call. The optional browser cognition layer can drive a draft/review/revise cycle, while bridge reflection runs after the final response. The pipeline is intentionally not presented as independent general intelligence: it is an orchestration sequence with bounded prompts, provider calls, and heuristics.

4. Memory: ordinary, protected, and contextual

4.1 Ordinary memory

SQLite is the default durable memory backend. The bridge can also use Azure Data Explorer (Kusto) when configured. The memory subsystem supports lexical recall, synonym expansion, optional embeddings with a persistent cache, source conversation/turn identifiers, and reflection-derived entities and emotion state. The user-facing Memory view is intended to show records and corrections rather than hide a permanent prompt transcript.

The lifecycle is conservative:

1. A deterministic fact path recognizes an explicit user-stated fact.
2. The bridge commits the fact before the acknowledgement is rendered.
3. Normal questions and mixed fact-plus-request messages do not automatically become facts.
4. Later recall is request-scoped and can be edited, corrected, or excluded.
5. Reflection and background jobs may derive or propose summaries and entities. The current background tick applies its proposals automatically and records applied or failed outcomes; this must not be confused with a universal human-approval boundary.

A structured memory record separates its primary **Value** from its explanatory **Context**. Value captures the fact, preference, constraint, or decision; Context supplies why it was retained, when it applies, or how its meaning should be interpreted. Context is editable, persists through correction, and accompanies recalled

user-profile and scenario records. It is historical data subject to the same prompt-framing precautions as the value, not a new channel for authoritative instructions. Existing records can carry conservative provenance-based context rather than invented motives; richer explanations require review.

This is a useful trust boundary, not a truth oracle. Extraction can be incomplete; lexical and semantic recall can miss relevant records or surface a nearby one; a user can later correct an entry; and model reasoning over recalled text can still be wrong. Context should carry provenance and confidence, and the model should treat it as evidence to check.

4.2 Protected memory

Protected memory is physically and logically separated from the ordinary database. The bridge creates a protected vault under the configured runtime root and exposes locked-safe metadata. Values are eligible for prompt release only when the vault is unlocked, model release has been approved for the session, and the current message matches a record category or public label. Artifacts are never injected into prompts. Government identifiers, financial, medical, document, and general categories are treated differently from ordinary memory.

The separation matters operationally: a local application can still be compromised, a model can still be induced to reveal context, and a user can still approve an unsafe release. Protected memory reduces accidental exposure and makes release a deliberate state; it is not a substitute for host security, key management, provider policy, or user judgment.

4.3 Context trust lifecycle

Eva's useful unit is not “memory” alone but a context lifecycle:

```
statement -> candidate -> committed record -> recalled evidence
      -> user correction / expiry / suppression
      -> reflection or consolidation proposal
      -> applied summary / recorded failure
```

The lifecycle should preserve the difference between source text, extracted atom, model interpretation, and a proposed summary. A confidence score or topology edge can support inspection, but it must not be rendered as proof. The current repository has an evolving heuristic and structured-memory design; the whitepaper does not claim a complete neural graph, perfect autobiographical continuity, or lossless recall.

4.4 Background consolidation

Background consolidation compresses conversation evidence and derives memory summaries through bounded jobs. The current background tick uses hands-off application: every job proposal is passed to its application handler, irrespective of its legacy `auto_apply` hint, and its outcome is recorded as applied or failed. Approval/rejection endpoints remain for pending proposals and compatibility, but do not describe the normal tick's approval policy.

This is an important governance tradeoff. Auditability is not equivalent to prior consent, and an automatically applied summary can still omit context or misinterpret evidence. Consolidation previews, selectable approval policies, and stronger correction propagation are directions for improvement, not guarantees of the current implementation.

5. Native actionability and receipts

Eva's action surface spans:

- browser and desktop automation, with confirmation for sensitive actions;
- bounded document operations for DOCX, PDF, PPTX, XLSX, and MCP Builder abilities;
- live retrieval for weather, news, markets, and configured web search;
- Signal and local mail integrations where separately configured;
- Skills imported from text, URLs, repositories, or files and normalized as untrusted data;
- coding workspaces, Git worktrees, terminals, assets, and agent controls;
- optional camera/presence sensing and local voice.

A native action should produce a receipt with the strongest result it can verify. Stock quotes use a local receipt or bounded provider path and report unavailable rather than asking a model to guess. Email distinguishes drafted, submitted, transport-failed, deferred, handed-off, and finally verified states. GitHub status replies rely on an immediately preceding verified issue URL or explicit failure language. This style prevents the common failure mode in which a fluent model says “done” although no side effect occurred.

Consent is layered rather than binary. Routine standing consent can cover read, search, fetch, and think in appropriate interactive paths. Execute, edit, move, delete, remote mutation, opaque execution, and unknown operations are evaluated under the active policy, not one universal prompt rule. The workspace composer defaults its **Auto approve actions** option to enabled; workspace autonomy can authorize permitted edits, builds, package operations, Git, GitHub, and remote-tool actions without a fresh interactive prompt. Outside-worktree edits and protected-path commands remain hard-blocked. Operators should review this default and choose a policy appropriate to the sensitivity of their work.

6. Workspaces, terminals, Skills, and media

6.1 Workspaces and isolated runs

The workspace control room presents projects, runs, terminals, live status, details, and Assets. Eva can create or import local Git workspaces, register projects, create managed worktrees, dispatch agents, interrupt or direct a run, inspect checkout status, and open run-relative files. Renderer DTOs contain opaque IDs and relative paths rather than absolute paths. The bridge revalidates managed paths and rejects traversal, symlink substitution, missing/non-regular files, and disposed runs.

The current model is “Eva control plane + explicit execution broker + replaceable runtime adapters.” ACP is the first workspace runtime. A future adapter is expected to provide structured lifecycle events, a per-worktree working directory, interceptable permissions, cancellation, and resumable sessions; a TUI's painted output alone is not a sufficient protocol.

6.2 Electron terminal boundary

The Electron main process owns `node-pty`, registers allowlisted roots, and exposes narrow preload functions for list/create/replay/write/resize/close. The renderer cannot provide an arbitrary working directory or environment. Shutdown escalates process descendants before root revocation. This is a strong boundary for a desktop app, but Electron's main process, Node dependencies, host OS, and user account remain trusted computing base components.

6.3 Skills

Skills are reusable instructions matched by semantic similarity and injected into context. The library supports provenance-aware browsing, Active/Draft/Disabled lifecycle, import, editing, and auto-learned drafts. Imported source text is untrusted and must pass anti-injection handling. A Skill is not an executable authority; the action policy still governs any tools it requests.

6.4 Voice, camera, and documents

Optional local voice dependencies provide Chatterbox synthesis, Faster Whisper transcription, and Silero VAD through a token-protected loopback service. Microphone audio is not uploaded to a cloud transcription service by that local voice path. Camera discovery and presence sensing are optional, off by default, and subprocess-isolated. Document abilities run locally through the bridge and remain bounded; they do not silently fall back to browser, desktop, terminal, or network actions when a supported operation is unavailable.

7. Deployment and operational model

7.1 Browser versus desktop

The browser is the lightweight UI and can use a separately managed bridge. Electron packages the UI with a locally started bridge, privileged main-process IPC, workspace terminal support, and optional local voices. The desktop package is therefore more capable but has a larger trusted surface and stronger host assumptions.

The documented development baseline is Node.js 24+, Python 3.12+, and an authenticated Copilot CLI for ACP-backed features. Direct OpenAI and LM Studio chat do not require Copilot. Linux Appliance is the proving ground; the inspected native terminal module references `GLIBC_2.34`, so the package is not an older-Linux compatibility promise. Python, Node, Copilot, compatible glibc/desktop libraries, and optional speech dependencies can remain external prerequisites. x86_64 and arm64 support should be stated only for artifacts and CI that actually verify those targets; it should not be inferred from a generic Appliance label.

7.2 Split ACP prerequisites

The repository records a split-deployment state in which the static web tier can run on legacy 32-bit hosts while the ACP bridge runs on a separate compatible machine. A future single-host deployment requires new hardware, Node/Python baseline installation, Copilot authentication, a systemd service, `/health` validation, an AIG smoke test, and the relevant Eva validation bundle. Until that milestone is completed, localhost fallback and explicit deployment boundaries are part of the architecture.

7.3 Endpoint and network restrictions

The bridge defaults to `127.0.0.1`. Workspace routes require an additional capability held only by Electron main. MCP credentials are redacted from API responses and persisted configuration. Web fetching validates DNS, rejects private-network destinations, pins validated addresses, rechecks redirects, and does not bypass sign-in or bot challenges. LM Studio URLs are restricted to localhost/private addresses on an allowlist of ports. These are defense-in-depth controls, not a claim that a compromised host or malicious provider is harmless.

8. Technical analysis

8.1 Performance

Eva adds orchestration work before and after model generation: memory lookup, possible embeddings, data retrieval, cognition calls, provider translation, reflection, and rendering. AIG streams only the final responder; internal memory, ACP preflight, and cognition calls remain non-streaming. This keeps the user-visible stream coherent but can increase time to first token. Bridge telemetry records TTFT, completion time, total timing, chunk count, route, and model labels without content, creating a basis for later measurement.

Performance is workload-dependent. Local lexical recall can be inexpensive; embedding cache misses and Kusto calls add network latency; ACP adds CLI process/session overhead; local models trade network latency for hardware throughput; terminal replay and workspace file projection are bounded to avoid unbounded renderer payloads. The current evidence does not justify published latency or throughput numbers.

8.2 Failure and recovery

Failure is represented at multiple layers: provider unavailable, MCP unavailable, partial retrieval, missing receipt, rejected permission, bridge crash, terminal load failure, workspace path invalidation, and transport uncertainty. The correct response is not always retry. A read can often retry; a remote mutation needs idempotency and a receipt; a mail submission may be accepted by a local MTA while final delivery remains unknown.

Electron retries a bridge port on address collision and waits for health. A bridge crash surfaces an error and quits rather than leaving a half-connected desktop. Workspace dispatch is intended to be idempotent/recoverable, while terminal metadata restoration after an Electron restart is still limited: renderer refresh reconnect is implemented but live metadata is not fully restored from `terminal_sessions`.

8.3 Concurrency and state

The bridge uses threaded HTTP handling and background reflection/consolidation paths. Shared state such as SQLite memory, protected-vault access, embedding cache, provider keys, and workspace records needs locks, session identifiers, and bounded writes. Conversation IDs and turn IDs scope memory operations; telemetry intentionally avoids transcript content. Concurrency creates real tradeoffs: reflection can finish after a response, two tabs can request conflicting edits, a run can be archived while an agent reports, and a bridge restart can interrupt a process. Durable state and explicit run status are more reliable than assuming process liveness.

8.4 Privacy boundaries

Local state is not automatically private. SQLite databases, protected vaults, browser profiles, logs, model caches, workspace files, provider APIs, MCP subprocesses, and the host OS each have different exposure. Eva's repository policy keeps secrets and runtime data out of source and release artifacts, but deployment owners must still protect the runtime root, OS account, backups, and configured external services. Cloud routes necessarily transfer selected context to their provider. The local mode reduces that transfer; it does not make network-enabled MCP retrieval offline.

8.5 Electron trust limits

The preload API is intentionally narrow, but Electron main can access the filesystem, spawn processes, and handle credentials. A renderer exploit would be serious even with capability checks. The application reduces blast radius with context isolation, opaque projections, path confinement, loopback tokens, and approval policy; it cannot replace timely Electron, Node, OS, and dependency updates.

9. Differentiation without hype

Eva overlaps with several categories:

Category	Typical strength	Eva's architectural answer	Tradeoff
Generic chat assistant	Fast conversation and broad model access	AIG routing plus inspectable memory and native receipts	More policy and state machinery than a simple chat client
Local model UI	Data locality and control	LM Studio route, SQLite, local voice, loopback bridge	Local quality, hardware, and tool availability vary
Coding-agent CLI	Repository changes and terminal fluency	Workspaces, managed worktrees, terminal broker, Assets, approval	ACP is first adapter; some lifecycle features remain future
Cloud agent platform	Hosted tools and collaboration	Optional cloud adapters behind a local control plane	Cloud features remain provider-dependent and may leave the machine
Automation/RPA tool	Deterministic UI actions	Consent, receipts, browser/desktop agents, native data tools	Model interpretation can still be uncertain; policy is essential

Category	Typical strength	Eva's architectural answer	Tradeoff
Memory framework	Structured recall and personalization	SQLite/Kusto atoms, provenance, corrections, protected vault	Heuristics are incomplete; no claim of perfect personal model

This is a design comparison, not a benchmark. Eva has no substantiated competitive accuracy, latency, safety, or cost advantage in this paper. Its defensible distinction is the combination of local ownership, an explicit execution broker, model interchangeability, and an operational UI that treats context and evidence as inspectable objects.

10. Goals and staged direction

Roadmap claims below are direction, not completed functionality.

Stage A — make the current foundation legible

Goal: stabilize the shipped browser/bridge/Electron contracts.

- Keep AIG, direct providers, ACP, LM Studio, streaming, memory endpoints, and approval behavior compatible.
- Add operator-facing diagnostics for route, capability, consent, receipt, and partial-data states without exposing secrets or prompt text.
- Restore terminal metadata after restart and document platform-specific native validation.
- Define measurable reliability and latency baselines using sanitized fixtures.

Verification: route contract tests, static/security checks, restart recovery scenarios, receipt fixtures, and telemetry reports that contain no user content.

Stage B — durable work orchestration

Goal: make a coding run recoverable and reviewable across process restarts.

- Complete provider-neutral adapter interfaces with structured lifecycle events, cancellation, resumable sessions, and per-worktree `cwd`.
- Add durable approval/audit UI, diff review, patch export, and explicit apply/commit/merge decisions.
- Add multi-agent sibling worktrees only after conflict, quota, and cleanup semantics are specified.

Verification: crash/restart matrices, idempotent dispatch tests, path/symlink adversarial fixtures, conflict recovery, and human review of every irreversible transition.

Stage C — evidence-aware memory and research

Goal: improve recall without turning context into authority.

- Make atom-to-source and source-to-summary lineage queryable.
- Add user-visible confidence explanations, expiry/suppression, correction propagation, and consolidation previews.
- Evolve bounded research into a durable job with checkpoints, source deduplication, restart continuation, and evidence-aware synthesis.

Verification: held-out synthetic conversations, recall precision/coverage measurements, false-commit and false-recall rates, provenance completeness, and red-team tests for prompt injection. These are proposed evaluations, not achieved product results.

Stage D — deployment breadth and adapters

Goal: reduce host and provider lock-in without weakening boundaries.

- Complete single-host ACP deployment after hardware and service prerequisites are met.
- Validate x86_64 and arm64 artifacts independently, plus macOS/Windows native terminal packaging where supported.
- Evaluate Cline, Gemini CLI, Goose, Codex CLI, and other adapters only where structured lifecycle and permission contracts exist.
- Complete or explicitly retire experimental Gmail/Outlook, Work IQ, remote MCP mailbox, browser evidence, Eva Field, and Sessions migration work.

Verification: reproducible artifact manifests, platform CI, `/health` and AIG smoke checks, adapter conformance tests, and documented data-transfer inventories.

11. Proposed evaluation program

Eva should earn trust through observable scenarios rather than a single “intelligence” score. The following program is intentionally prospective.

1. **Routing correctness:** classify a corpus of ordinary chat, current-data, tool-required, local-only, and mutation requests. Measure correct route, fail-closed rate, unnecessary cloud transfer, and user override fidelity.
2. **Memory quality:** measure explicit-fact precision, recall coverage, correction latency, stale-fact suppression, provenance completeness, protected-release violations, and summary approval accuracy. Include adversarial ambiguous statements.
3. **Action integrity:** for each native action, compare claimed completion with a verified receipt. Measure false-success, duplicate mutation, authorization bypass, partial-result clarity, and cancellation latency.
4. **Workspace safety:** run traversal, symlink swap, outside-root, process-tree, crash, restart, and concurrent-dispatch fixtures. Require no unauthorized file or process access.
5. **Operational performance:** report TTFT, end-to-end latency, memory lookup latency, retrieval latency, stream interruption recovery, CPU/RAM, and provider-specific failure rates by route and hardware class.

6. **Privacy review:** inventory every context transfer, inspect logs for secret/content leakage, test bridge token and workspace capability isolation, and review provider/MCP subprocess environments.

No benchmark numbers, certification, or compliance status should be inferred until these evaluations are run, versioned, and reproducible.

12. Material limitations and risk register

- **Memory is incomplete and editable.** Heuristics, embeddings, synonym expansion, and summaries can miss, misclassify, or stale information. Context is not authority.
- **Protected memory still has a trust boundary.** Unlock and release decisions, host security, vault keys, and provider behavior remain consequential.
- **External providers can fail or change.** Copilot authentication, OpenAI availability, LM Studio model behavior, MCP servers, web sources, Signal, Kusto, and local mail are outside Eva's complete control.
- **Guardrails can fail.** Prompt injection, malicious Skills, compromised MCP tools, model misclassification, and confused-deputy paths require layered controls and testing.
- **Receipts have scope.** A receipt can prove that a local submission occurred without proving final external delivery or the correctness of the resulting content.
- **Applimage is not a universal appliance.** Host glibc, desktop libraries, native modules, Node/Python, Copilot, and optional voices affect capability.
- **Research is bounded.** One search and at most two public page reads in the current first slice are not exhaustive research, and “complete” means the bounded batch finished.
- **Electron is privileged.** A desktop compromise can reach more than a browser compromise; narrow IPC reduces but does not eliminate risk.
- **Concurrency is real.** Background reflection, multiple tabs, provider retries, run recovery, and process restarts can expose races unless state transitions are durable.
- **The visual metaphor is limited.** Confidence/connectivity topology is a UI aid, not a true 4D neural brain graph, cognitive proof, or evidence of sentience.
- **Experimental integrations are not promises.** Gmail/Outlook OAuth, Work IQ, remote MCP mailbox paths, broader adapter support, Eva Field, and several review workflows are not presented as shipped.

13. Evidence appendix

The following links are relative to this repository and identify the primary evidence used for this paper. Line numbers are intentionally omitted where files are living documents; the linked sections and source names are the durable anchors.

Product positioning and user-facing behavior

- [README.md](#) — local-first positioning, highlights, current development versus latest published release, Trust by Design, and documentation index.
- [README-2.md](#) — setup, provider modes, memory, retrieval, architecture, streaming, workspaces, security, attribution, and roadmap.
- [docs/ai-development-guide.md](#) — ownership map, contracts, validation boundaries, and explicit warnings not to inspect runtime/private data.

AIG and bridge

- [core/js/providers/aig.js](#) — browser AIG lifecycle, explicit-fact handling, receipts, and response orchestration.
- [tools/bridge/aig_request.py](#) — request normalization, route policy validation, continuation handling, and action-status receipt logic.
- [tools/bridge/core.py](#) — ACP bridge, backend parsing, loopback OpenAI-compatible endpoint restriction, token limits, and local response handling.
- [core/js/cognition.js](#) — optional browser-side cognition pipeline and action execution boundary.

Memory, protection, and state

- [tools/bridge/memory.py](#) — SQLite/Kusto selection, ordinary recall, embeddings, protected-memory context gating, and backend lifecycle.
- [tools/protected_memory.py](#) — protected vault implementation.
- [tools/sqlite_memory.py](#) — local structured memory persistence.
- [tools/bridge/cognition.py](#) — bridge cognition/reflection support and bounded memory-related processing.

Desktop, workspaces, and capability boundaries

- [standalone/main.js](#) — bridge lifecycle, Electron main-process trust boundary, terminal broker initialization, IPC validation, and workspace capabilities.
- [standalone/preload.js](#) — narrow renderer-facing terminal, workspace, voice, and authentication APIs.
- [standalone/terminal-broker.js](#) — PTY ownership and lifecycle boundary.
- [tools/bridge/workspaces.py](#) — workspace routes, run lifecycle, path confinement, and asset projection.
- [core/js/features/workspaces/monitor.js](#) — workspace monitor behavior and user-facing run controls.

Packaging and skills

- [standalone/README.md](#) — ApplImage runtime compatibility, update behavior, download setup, and host prerequisites.

- [tools/bridge/skills.py](#) — Skill import, matching, and lifecycle behavior.
- [docs/eva_default_skills/README.md](#) — default catalog and category taxonomy.

The appendix is evidence for architecture and stated behavior, not a certification report. The commit and date at the top of this document should accompany any future revision so that claims can be rechecked against the source snapshot.

14. Implementation status at the evidence snapshot

The distinction between “implemented,” “experimental,” and “roadmap” is central to reading this paper. The table below is a compact status register, not a promise that every feature has equal maturity or platform coverage.

Capability	Status at 8e13559	What that means
Browser UI, settings, AIG conversation, response rendering	Implemented	The framework-free UI and provider-facing lifecycle are present in the repository.
Python bridge and OpenAI-compatible chat contracts	Implemented	Local HTTP endpoints, normalization, streaming, health, and provider paths are present; deployment still has prerequisites.
Automatic/pinned AIG routing	Implemented	Policy modes and route validation exist; model quality and availability remain provider-dependent.
Direct OpenAI, Copilot ACP, LM Studio	Implemented / route-dependent	Each path is wired, but credentials, local services, authentication, or tool support may be required.
SI Harness naming and OpenAI-compatible endpoint surface	Implemented as product architecture	Compatibility is a contract and orchestration surface, not an AGI or uniform-capability claim.
SQLite memory, optional Kusto, provenance and corrections	Implemented	Durable local memory and optional cloud-backed memory are supported, with known recall limitations.
Protected memory vault and gated release	Implemented	The separate vault and release states exist; host and key security remain out of scope.
Reflection, emotion, consolidation proposals	Implemented foundation / bounded	Background processing and proposal concepts exist; they do not imply complete self-modeling or autonomous learning.
Native receipts and consent paths	Implemented in selected actions	Coverage is action-specific; a receipt can be partial or unavailable and should not be generalized.
Workspaces, worktrees, terminal broker, Assets	Shipped foundation	The operational surface exists, while restart metadata, adapter breadth, and review workflows have gaps.
Skills catalog and bounded document abilities	Implemented foundation	Import, matching, lifecycle, and selected document operations exist; imported content remains untrusted.

Capability	Status at <code>5e13559</code>	What that means
Local voice, camera, browser and desktop automation	Optional integrations	They require platform, dependency, permission, or user configuration and are not universal defaults.
Bounded online research	Implemented first slice	Current behavior is intentionally limited and is not a durable deep-research engine.
Gmail/Outlook OAuth, Work IQ, remote mailbox MCP	Experimental or disabled	Groundwork must not be represented as a complete connection lifecycle.
Provider-neutral multi-agent adapters, durable approval/audit UI, diff review, explicit merge	Roadmap	These are direction and design constraints, not shipped general capabilities.
Single-host ACP, broader macOS/Windows packaging, Eva Field, Sessions migration	Roadmap or platform work	The repository documents prerequisites and milestones that remain to be completed.

This register also explains why the development version must be qualified. The README records 5.6.12 as unreleased and 5.6.10 as the latest published standalone release. The evidence snapshot therefore describes a moving development product, not a generally available release with a completed support matrix. The AI development guide is itself marked as last reviewed against an earlier 5.6.2 baseline; its ownership and safety guidance is useful, but readers should not treat that review date as proof that every newer feature has been independently audited.

15. Governance and adoption guidance

Technical adoption should begin with a capability inventory rather than a model preference. An organization should decide which data may remain on the workstation, which providers may receive context, which MCP servers are approved, which paths are eligible for workspaces, and which actions require a person every time. Those decisions should be recorded beside deployment configuration, not inferred from a model selector.

15.1 Recommended operating profiles

Local evaluation profile. Use SQLite, LM Studio, loopback-only bridge access, no external MCP servers, and local voice only if needed. Start with synthetic or low-sensitivity repositories. This profile tests UI, memory inspection, workspace confinement, and receipts without making a cloud-transfer assumption.

Hybrid knowledge-work profile. Permit selected web/MCP retrieval and a direct provider for tasks that benefit from current information, while retaining local memory and explicit approval for mutations. Record the provider and retrieval timestamp in operational review.

ACP coding profile. Use authenticated Copilot ACP only for selected workspaces. Keep terminal capability under Eva's broker, require interactive approval for external mutations, and treat the worktree and Assets views as the review surface. Do not infer that a successful ACP session means a patch is safe to merge.

Protected-data profile. Keep sensitive records in the protected vault, disable model release by default, and test category/label matching with synthetic fixtures. A user should be able to explain why a protected value was released in a particular session.

15.2 Change control

Provider adapters, memory schema changes, and new native actions should be reviewed as contract changes. The relevant questions are:

- Does the browser still preserve the AIG top-level selector and expected response lifecycle?
- Can a request reach a provider or tool that the policy did not authorize?
- Does a failure leave a durable state that can be reconciled after restart?
- Can a model-generated sentence be mistaken for a verified receipt?
- Are secrets, runtime data, absolute paths, and private URLs absent from logs, fixtures, documentation, and artifacts?
- Can an operator disable the capability without deleting unrelated memory or workspace state?

This governance model favors smaller, reversible changes. It also acknowledges a product truth that broad feature lists can conceal: reliability depends on the interaction of UI, bridge, provider, host, network, and user decisions. A capability should be called shipped only when its owner, failure states, platform assumptions, and verification path are documented.

15.3 What sophisticated adopters should measure first

Before expanding scope, an adopter should establish a baseline with sanitized workloads: median and tail response latency by route; retrieval and reflection overhead; rate of unverified action claims; bridge restart recovery; workspace path-rejection coverage; and the proportion of requests that leave the approved network boundary. These measures are more decision-useful than a generic model leaderboard because they test Eva's differentiating contracts. Results should be tied to a commit, provider version, host class, and configuration so that a later change can be explained rather than attributed vaguely to "the AI."